

Statistical Learning with High-dimensional Data



Pr. Charles BOUVEYRON

Professor of Statistics and Artificial Intelligence
Chair of the Institut 3IA Côte d'Azur
Deputy Scientific Director of the Institut 3IA Côte d'Azur
Head of the Maasai research team
Université Côte d'Azur & Inria

✉ charles.bouveyron@inria.fr - [🐦 @cbouveyron](https://twitter.com/cbouveyron)

Logistic regression :

LR is a method for binary classification ($C=2$) and it is probably the most used for this task in real-world problems. This method is also a reference method.

LR gets its name from the fact it transforms a regression problem in a classification one thanks to the logistic function:

$$\begin{aligned} \text{logit} \left(P(Y=1 | X; \theta) \right) &= \log \left(\frac{P(Y=1 | X; \theta)}{P(Y=0 | X; \theta)} \right) \\ &= \beta_0 + \beta^T X + \epsilon \quad \text{with } \epsilon \sim N(0, \sigma^2) \end{aligned}$$

In practice :

(i) estimate $\Theta = \{\beta_0, \beta, \sigma^2\}$ where β is a vector.
by Maximum Likelihood.

! Unfortunately, and contrary to linear regression, there is no closed-form solution to this problem and we have to use an optimization algorithms (SGD, ...)

(ii) $\hat{J}$ is then simply $\text{MAP} \rightarrow \hat{\Theta}$

$$\hat{J}(x^*) = \arg\max_c P(Y=c/X=x^*; \hat{\Theta})$$

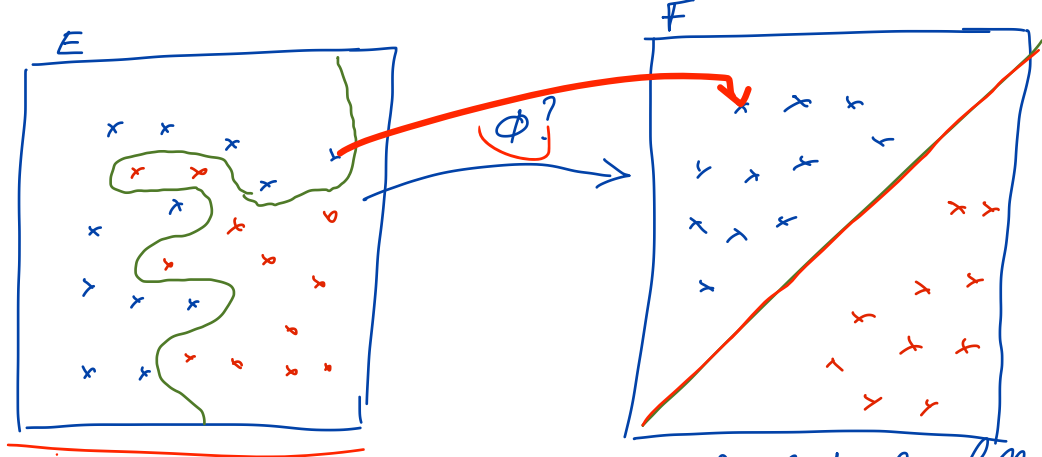
$$P(Y=1/X, \hat{\Theta}) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}^T x)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}^T x)}$$

$$\geq 0.5 \Rightarrow \hat{y}^* = 1$$

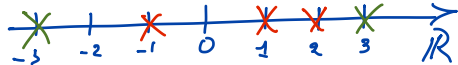
Support Vector Machines :

SVMs is a family of ML methods that relies on the idea of projecting the data in a higher dimensional space in order to simplify the classification problem.

The idea of SVM is to find a non-linear projection ^{operator} ϕ that will project the low-dim. data $X \in E$ in a higher-dimensional space F (the feature space) such that the classification problem can be solved linearly in F .



Exercise: Find an operator ϕ such that the following classification problem can be solved linearly:



$$\phi(x) = x^2$$

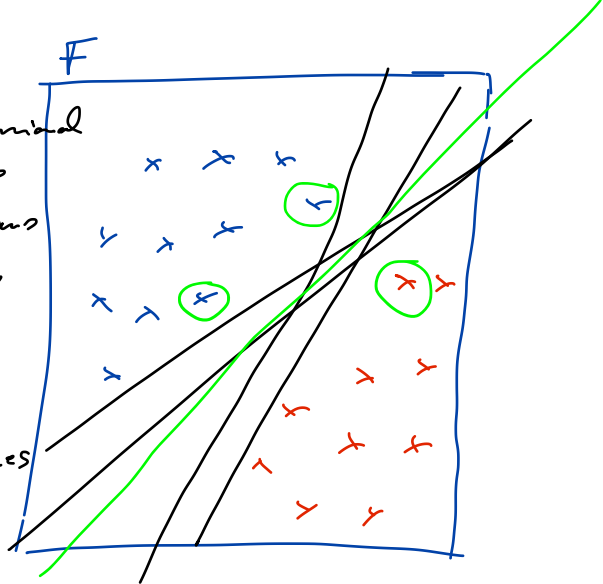
$$\phi(x) = |x|$$

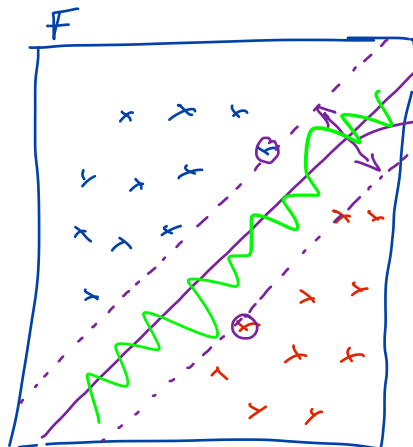
In practice, the goal of SVM is even more strict in terms of the choice of the linear separator: SVM looks for a classification boundary in the feature space that is the most far away from all data. The goal of this constraint is of course to avoid overfitting.

⇒ Definition: the data points of the feature space that are the closest ones to the classification boundary are called the support vectors

Rule: in a high-dimensional feature space, the nb of candidate hyperplanes that separates linearly the data is going to infinity.

$\Rightarrow$ only one maximizes the distance to all data.





H^* is the hyperplane that maximizes the margin between the support vectors.

$\Rightarrow$ The search for the hyperplane H^* can be therefore transformed as an optimization problem:

$$\underbrace{H^*}_{H} = \underset{H}{\operatorname{argmax}} \mathcal{M}(\underbrace{H, \phi(x)}) \quad (1)$$

The nice mathematical idea of SVM has now to face with the difficulty of optimizing a function in a very high-dimensional space.

Thanks to the so-called "kernel ^{trick}", it is in fact possible to perform this optimization directly from the original data:

Definition: a kernel K associated with the operator ϕ is such that:

$$K(x, y) = \langle \phi(x), \phi(y) \rangle.$$

Thanks to this definition, it is possible to rewrite:

(1) $\Leftrightarrow$ $H^* = \arg \max_H \|\phi(x_1) - \phi(x_2)\|^2$ where x_1 and x_2
are the support vectors

but $\|\phi(x_1) - \phi(x_2)\|^2 = \langle \phi(x_1) - \phi(x_2), \phi(x_1) - \phi(x_2) \rangle$.

$$\begin{aligned} &= \underbrace{\langle \phi(x_1), \phi(x_1) \rangle} - \underbrace{\langle \phi(x_1), \phi(x_2) \rangle + \langle \phi(x_2), \phi(x_1) \rangle}_{\checkmark} \\ &\quad + \underbrace{\langle \phi(x_2), \phi(x_2) \rangle} \\ &= \underbrace{K(x_1, x_1)} - \underbrace{2K(x_1, x_2)} + \underbrace{K(x_2, x_2)} \end{aligned}$$

$\Rightarrow$ the optimization problem (1) can be therefore solved directly in E through the kernel K .

Naturally, the question of finding an operator ϕ has now become finding a kernel function K .

$\Rightarrow$ it is possible to define a very large number of different kernels, but there exists a list of good kernels:

$$x^t x'$$

- linear kernel : $K(x, x') = \langle x, x' \rangle$
- polynomial kernel : $K_d(x, x') = \langle x, x' \rangle^d$
- RBF kernel : $K_\gamma(x, x') = \exp\left(-\frac{1}{2\gamma} \langle x, x' \rangle^2\right)$

Remark: the choice of the kernel and/or its tuning parameters can be done using cross-validation.

Summary on SVM:

- ⊕ SVM is usually very efficient if we find the right kernel
- ⊕ SVM can be applied to any type of data if we are able to design a kernel.
- ⊖ The decision rule is difficult to visualize and understand.
- ⊖ Learning a SVM can be slow, in particular when n is large.

Outline

1. Introduction
2. Reminder of descriptive statistics
3. The learning process
4. Supervised learning
5. Unsupervised learning
6. Learning in high-dimensions
7. Dimension reduction
8. Clustering in high dimensions

Unsupervised Learning : a difficult task?

It is indeed a difficult task because we work with less information than in the supervised case, but some tasks are by nature unsupervised and it is not possible to escape this nature:

- image denoising
- autonomous cars.
- visualization
- ...

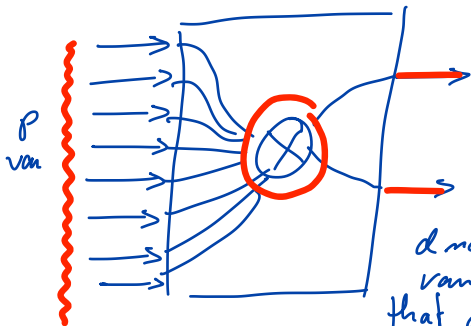
In this course, we will cover dimension reduction and clustering:

- dim. reduction : $X \in \mathcal{X}^p \xrightarrow{\text{learn}} Y \in \mathbb{R}^d$
where $d \ll p$
- clustering : $X \in \mathcal{X}^p \xrightarrow{\text{learn}} Y \in \{1, \dots, K\}$

5.1. Dimension reduction:

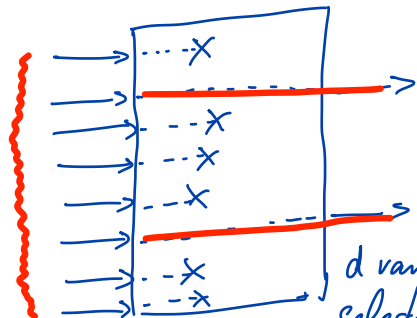
There are in fact two main families for this task:

(i) feature extraction



d new variables
that are combinations
of the p original
variables.

(ii) variable selection



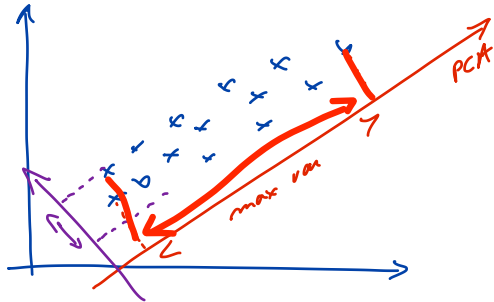
d variables
selected
as relevant among
the p original
ones.

⇒ feature extraction methods (PCA, PLS, ...) are really more efficient in term of information preservation, whereas feature selection methods are better for explainability.

⇒ feature extraction is also far faster than feature selection.

A quick refresher of PCA (and its probabilistic version)

The goal of Principal Component Analysis (PCA) is to build from the p original variables a set of d new variables, by linear combination, such that the projected data have a maximum variance.



PCA is therefore solution of an optimization problem:

$$Y = XU^* \text{ where } U \text{ is a } p \times d \text{ matrix}$$

such that $U^* = \underset{U}{\operatorname{argmax}} \text{ } \underline{\text{Var}(Y)}$

Theorem: The d first axes of U^* are the eigenvectors of $\Sigma = \overline{X}^T \overline{X}$ associated with the d largest eigenvalues of Σ (where $\overline{X}$ are the centered data).

In practice:

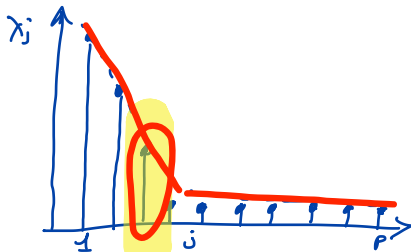
- compute the empirical covariance matrix $\Sigma = \overline{X}^T \overline{X}$
- do the eigendecomposition of Σ
- keep the eigenvectors associated with the largest eigenvalues.

Afterward, we also have to select the appropriate number of dimensions (d):

- the rule of 90% :

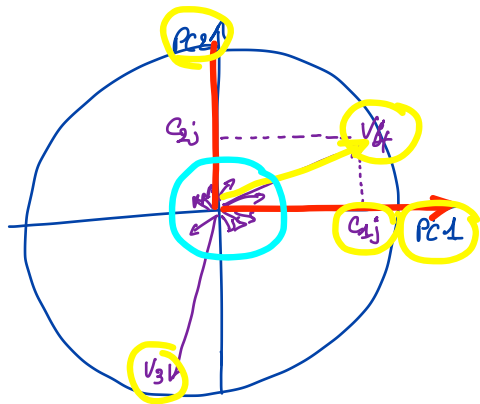
$$d^* = \underset{d}{\operatorname{argmin}} \frac{\sum_{j=1}^d \lambda_j}{\sum_{j=1}^p \lambda_j} \geq \underline{90\%}.$$

- the rule of the elbow in the eigenvalue scree :



$\Rightarrow \underline{d^* = 3}$ here.

In order to try to understand the relationships between the new PCA axes and the p original variables, it is possible to use the correlation circle:



where c_{ij} is the correlation between the i^{th} axis of PCA and the j^{th} variable

$$c_{ij} = \sqrt{\lambda_j \cdot u_{ij}^*}$$

u_{ij_star} is the coefficient of the projection matrix

where PC1 here means the first axis of PCA (i.e. the 1st column of U^*)

A few years ago, Tipping & Bishop (1996) proposed a probabilistic version of PCA: probabilistic PCA (PPCA)

PPCA assumes the following model:

Y: the smaller dimension version of X
or we call the latent variables

X: the observed variables

$$X \in \mathbb{R}^p = Y \cdot U^T + \epsilon$$

with

$$\epsilon \in \mathbb{R}^p \sim \mathcal{N}(0, \sigma^2 I_p)$$

and

$$Y \in \mathbb{R}^d \sim \mathcal{N}(0, I_d)$$

and U is a $p \times d$ transformation matrix.

Interestingly, the marginal distribution of X is:

$$X \sim \mathcal{N}(0, U U^T + \sigma^2 I_p)$$

It is then possible to use Maximum Likelihood to estimate the parameters of the model: $\theta = \{U, \sigma^2\}$

Here, the problem of PCA has been reformulated as a problem of statistical inference.

Theorem: the d first axes of U are the eigenvectors of $\Sigma = \bar{X}^t \bar{X}$ associated with the d largest eigenvalues.

! Thanks to Tipping & Bishop, we have a justification under a Gaussian assumption of PCA.

$\Rightarrow$ this model also opens the door to many new statistical methods designed for HD data.